This diagram also serves another purpose. Remember in Chapter 6, "Object Fundamentals," when we looked at interfaces for the first time? I mentioned dependency inversion. The relationship between `StockTaker` and `WarehouseInventory` is another example. Rather than have `StockTaker` dependent on the two inventory classes, we instead insert an abstraction: the `Inventory` interface. In this way the relationship between each inventory class and the `StockTaker` is "inverted" (the arrows point away from the classes and toward the interface). `StockTaker` only ever needs to check that an incoming object implements the interface; its type is irrelevant.

This is the second time we've encountered one of the SOLID principles, so let's move onto that next.

The SOLID Principles

SOLID is a set of five principles intended to aid the design of OOP software. They were introduced by Robert C. Martin, one in a 1996 paper ("The Single Responsibility Principle") and the other four in a 2000 paper ("Design Principles and Design Patterns"). The SOLID acronym was coined by Michael Feathers, and it stands for the following.

- **S**ingle Responsibility
- **O**pen Closed
- **L**iskov Substitution
- **I**nterface Segregation
- **D**ependency Inversion

Let's explore each of those principles in turn.

Single Responsibility Principle (SRP)

A class should do one thing and therefore (from Martin's 1996 paper) "should have only one reason to change." There is some ambiguity around the phrase "reason to change," and Robert Martin clarified in later writing that his intention was to point out the "coupling between the term 'responsibility' and 'reason to change'" and that responsibility was a question of to whom rather than to what. This principle concerns the relationship between actors (people or groups of people) and the system they use.